A primer on Convolutional Neural Networks

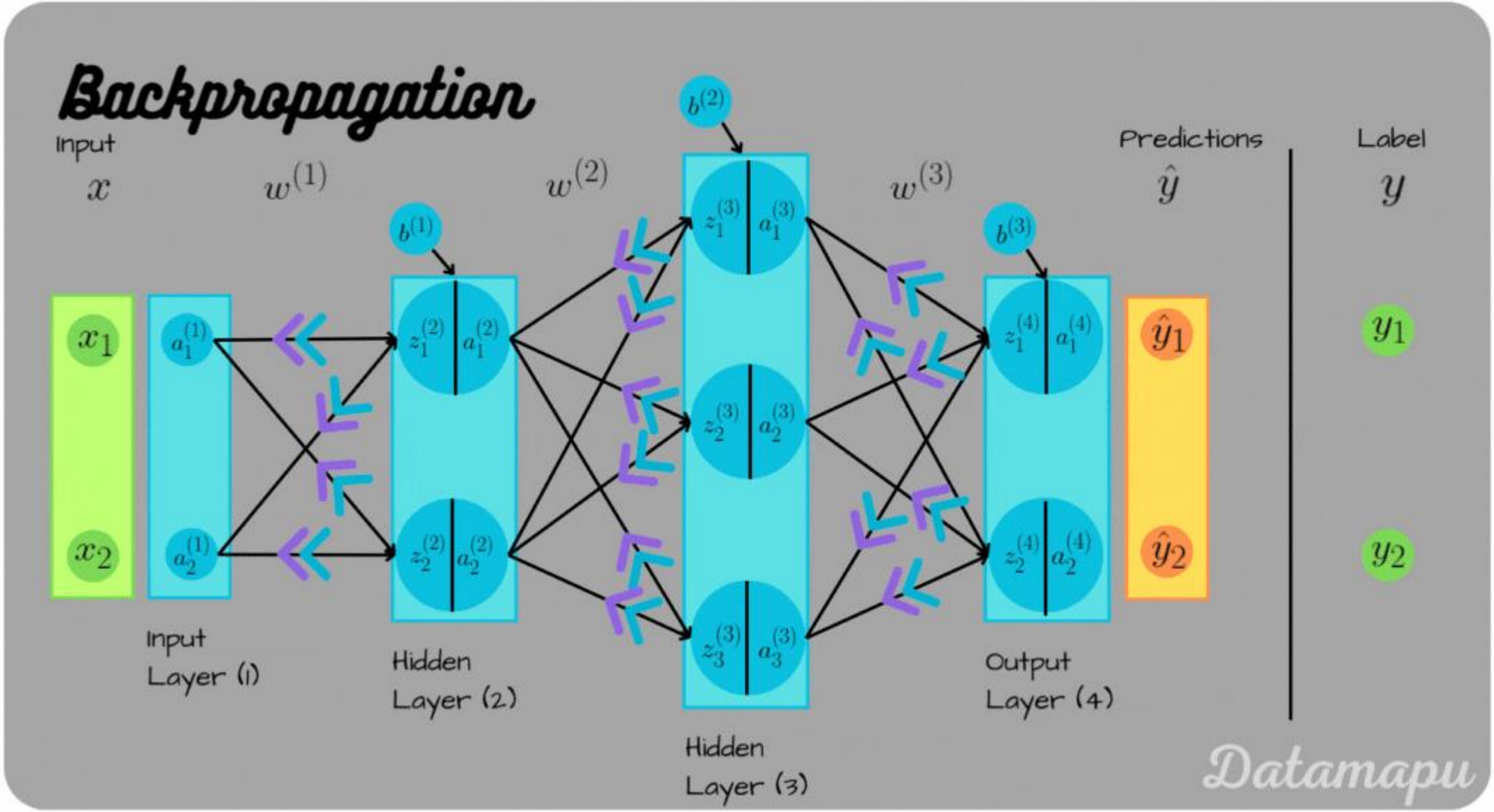
Biplab Banerjee

Multi-layer perceptron

- **What is a Multi-Layer Perceptron (MLP)?**
  - An MLP is a fundamental type of **feedforward artificial neural network (ANN)**.
  - It consists of at least three layers of nodes: an **input layer**, one or more **hidden layers**, and an **output layer**.
  - Each node, or neuron, is connected to every neuron in the next layer, which is why it's called "fully connected."
  - Its primary purpose is to model and learn complex, non-linear relationships in data.
- **The Power of Hidden Layers**
  - The key difference from a single-layer perceptron is the presence of **hidden layers**.
  - A single layer can only learn linearly separable patterns.
  - Hidden layers allow the MLP to learn and approximate any continuous function, enabling it to solve complex problems that are **not linearly separable**.
  - The number of hidden layers and neurons (the network's architecture) determines its complexity and learning capacity.

Multi-layer perceptron

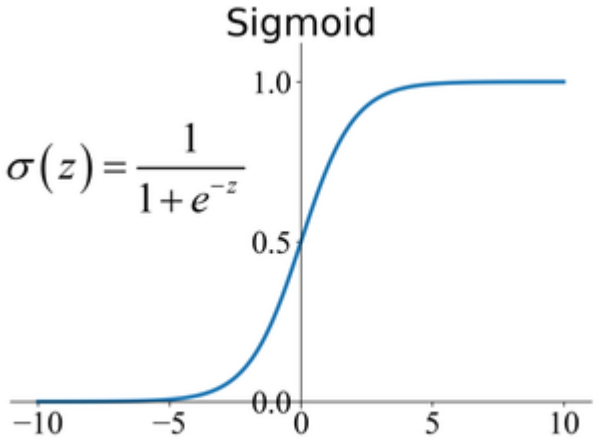
- **Activation Functions: Introducing Non-Linearity**
  - Every neuron in the hidden and output layers applies an **activation function** to its input.
  - The purpose of this function is to introduce non-linearity into the network. Without it, the MLP would behave just like a simple linear model, regardless of its depth.
  - Common activation functions include **ReLU (Rectified Linear Unit)**, Sigmoid, and Tanh. Each has specific properties suitable for different tasks.
- **How MLPs Learn: Backpropagation**
  - MLPs are trained using a process called **backpropagation**.
  - **Forward Pass:** Input data is fed through the network, layer by layer, to produce an output.
  - **Loss Calculation:** The network's output is compared to the actual target value to calculate an error or "loss."
  - **Backward Pass (Backpropagation):** The error is propagated backward through the network, and the connection weights between neurons are adjusted proportionally to how much they contributed to the error. This process is repeated iteratively to minimize the loss.



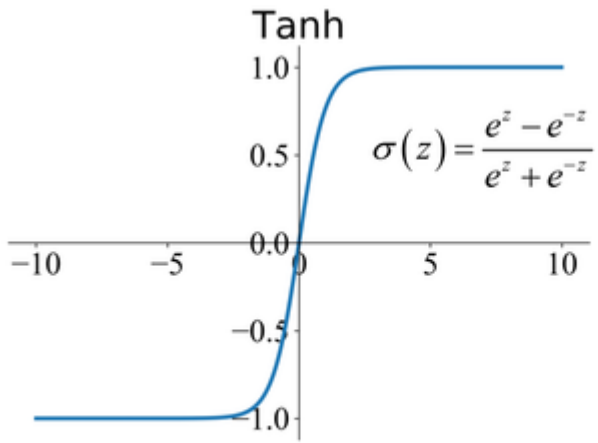
The Universal Approximation Theorem is a foundational concept in neural networks. In simple terms, it states that a feedforward neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function to an arbitrary degree of accuracy.

Imagine you have a complex, continuous curve you want to replicate. The theorem says that you can get incredibly close to matching that curve by using a neural network, provided you have enough neurons in its hidden layer. The neurons act like flexible building blocks, and with enough of them, you can build a structure that mimics the shape of any continuous function.

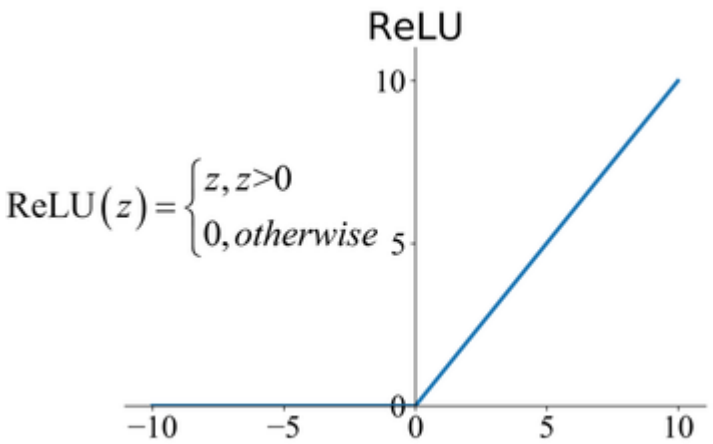
The activation functions for non-linearity



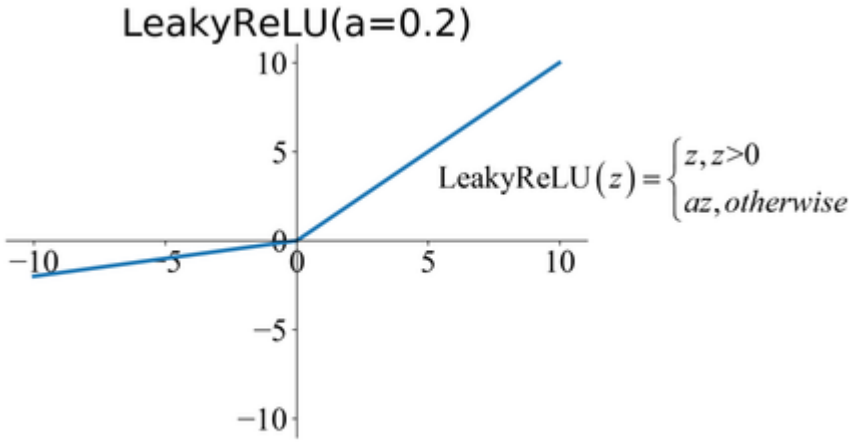
(a)



(b)



(c)



(d)

Gradients of the activation functions

1. ReLU (Rectified Linear Unit)

The ReLU function is $f(x) = \max(0, x)$. Its derivative is a piecewise function:

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$$

The derivative is technically undefined at $x = 0$, but in practice, it is set to 0. This constant 1 for positive values is what prevents the vanishing gradient problem.

2. Sigmoid

The Sigmoid function is $\sigma(x) = \frac{1}{1+e^{-x}}$. Its derivative is conveniently expressed in terms of the function's own output:

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

The maximum value of this derivative is 0.25 (when $\sigma(x) = 0.5$). Because the derivative is always a small number less than 1, multiplying it repeatedly during backpropagation causes the gradient to vanish in deep networks.

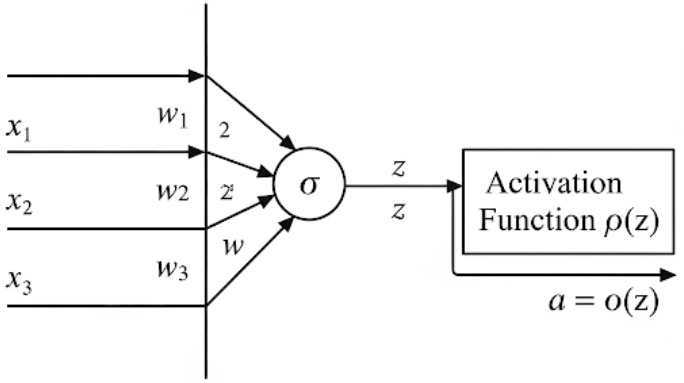
3. Tanh (Hyperbolic Tangent)

The Tanh function is $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Similar to the Sigmoid, its derivative can be expressed using the function's output:

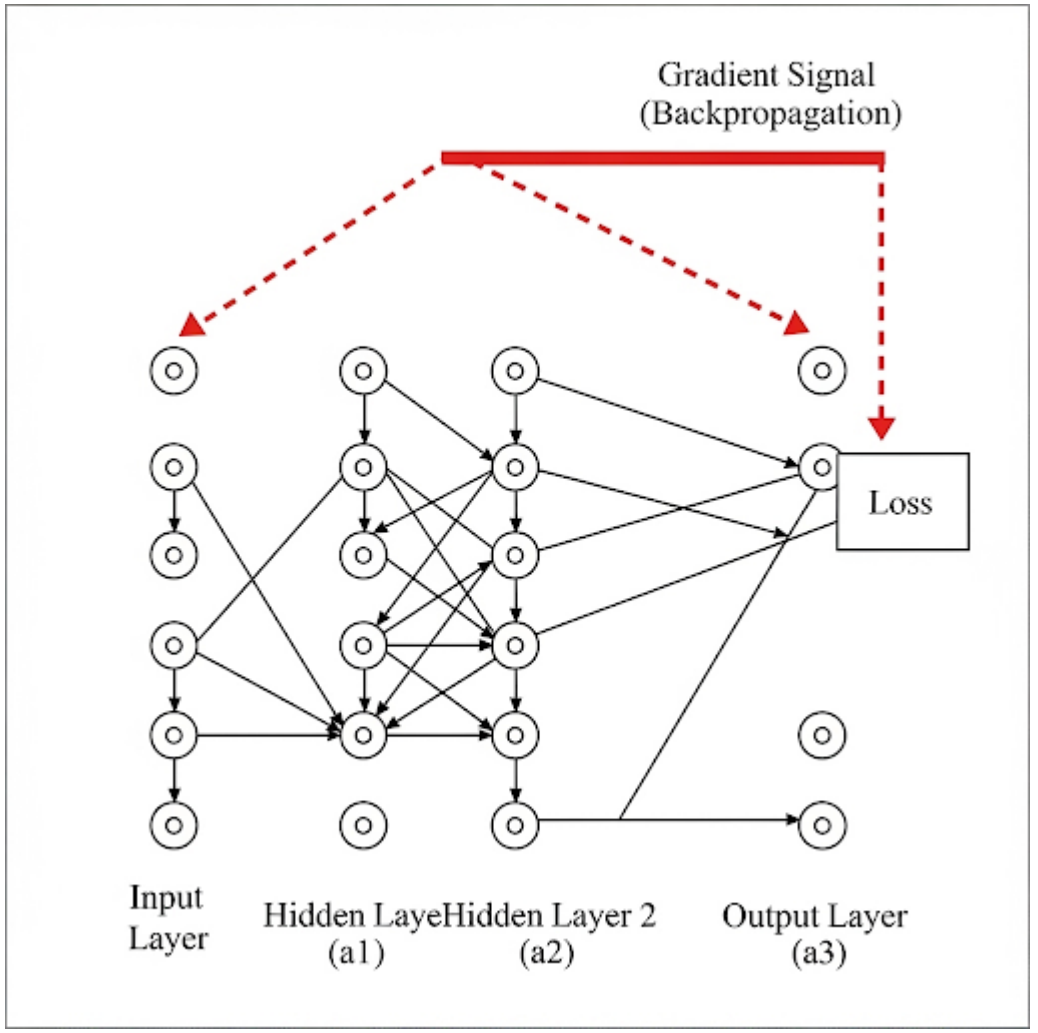
$$\tanh'(x) = 1 - \tanh^2(x)$$

The maximum value of the Tanh derivative is 1 (at $x = 0$), but it quickly drops towards zero for other values. While slightly better than Sigmoid, it still suffers from the vanishing gradient problem as the network gets deeper.

Chain rule insights



Outputs from a layer (or neuron) has weighted summation between inputs and weights followed by non-linearity



$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial a_3} \cdot \left(\frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_2}\right) \cdot \left(\frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1}\right) \cdot \left(\frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial W_1}\right) \qquad \frac{\partial L}{\partial W_1} \propto (\sigma'(z_3) \cdot W_3) \cdot (\sigma'(z_2) \cdot W_2) \cdot \ldots$$

Gradient of loss with respect to each layer weight using chain rule

Vanishing and exploding gradients

- During the training of a neural network (using backpropagation), the "gradient" is the signal used to update the network's weights. It tells the network how much to change a weight to reduce the error.
- **Vanishing Gradient:** This is when the gradients become extremely small as they are propagated backward from the output layer to the input layer. When the gradients are tiny, the updates to the weights in the early layers become minuscule. As a result, these layers learn very slowly or stop learning altogether.
- **Exploding Gradient:** This is the opposite problem. The gradients grow exponentially as they are propagated backward, becoming massive. This leads to huge, unstable updates to the weights, causing the model to fail to learn. The loss function might return NaN (Not a Number) as the values become too large to handle.

- The core reason lies in the **chain rule** used during backpropagation. The gradient for a weight in an early layer is calculated by multiplying the gradients of all the layers that come after it.

- Imagine a simple 4-layer network. The gradient for a weight in Layer 1 would look something like this:

  - Gradient (Layer 1) = Gradient (Layer 4) * Gradient (Layer 3) * Gradient (Layer 2)

  - **Vanishing occurs when** the gradients of the individual layers are consistently small (less than 1). For example, if the derivative of the activation function (like Sigmoid, as mentioned in your file) is always around 0.1, the final gradient would be 0.1 * 0.1 * 0.1 = 0.001. The more layers you have, the smaller it gets, effectively "vanishing.“

  - **Exploding occurs when** the gradients of the individual layers are consistently large (greater than 1). If the weight values are large, and the derivatives are also large, you might get something like 2.5 * 2.5 * 2.5 = 15.625. This value explodes as you add more layers.

Vanishing Gradient Example (with Sigmoid-like derivatives)

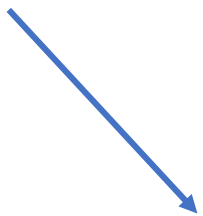
- Assume the derivative of the activation in each of our 3 layers is **0.1**.
- The error signal from the final layer is **1.0**.

1. **Gradient for Layer 3's weights:** `1.0 * 0.1 = 0.1`
2. **Gradient for Layer 2's weights:** `1.0 * 0.1 * 0.1 = 0.01`
3. **Gradient for Layer 1's weights:** `1.0 * 0.1 * 0.1 * 0.1 = 0.001`

Exploding Gradient Example (with large weights/derivatives)

- Assume the derivative/weight contribution from each of our 3 layers is **3.0**.
- The error signal from the final layer is **1.0**.

1. **Gradient for Layer 3's weights:** `1.0 * 3.0 = 3.0`
2. **Gradient for Layer 2's weights:** `1.0 * 3.0 * 3.0 = 9.0`
3. **Gradient for Layer 1's weights:** `1.0 * 3.0 * 3.0 * 3.0 = 27.0`



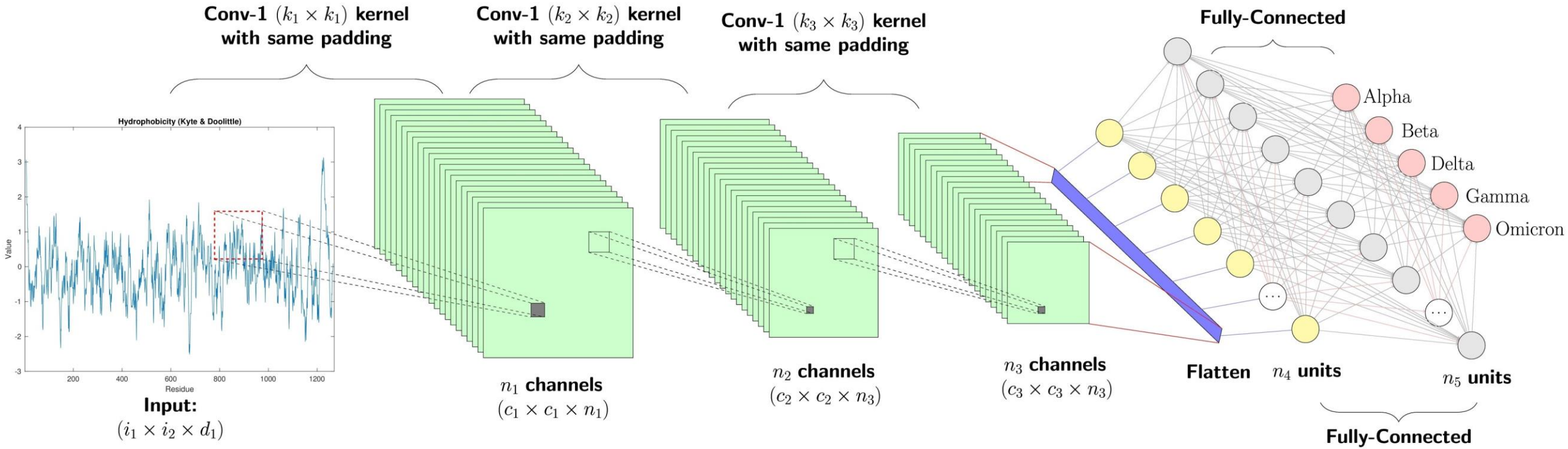
ReLU to the rescue!

1. **Gradient for Layer 3's weights:**
   `Error (1.0) * Derivative (1) = 1.0`
2. **Gradient for Layer 2's weights:**
   `Error (1.0) * Derivative (1) * Derivative (1) = 1.0`
3. **Gradient for Layer 1's weights:**
   `Error (1.0) * Derivative (1) * Derivative (1) * Derivative (1) = 1.0`

Moving from MLP to CNN

- **Loss of Spatial Information:** MLPs take flattened vectors as input. For an image, this means the 2D structure (or 3D for color images) is lost. All pixels are treated equally, regardless of their position relative to each other. This process discards crucial spatial hierarchies—the fact that pixels close together form edges, edges form shapes, and shapes form objects.
- **High Parameter Count (Inefficiency):** In an MLP, every neuron in one layer is connected to every neuron in the next. For a high-resolution image (e.g., 256x256 pixels), the input layer would have over 65,000 nodes. A hidden layer with a similar number of neurons would result in billions of parameters (weights). This makes the network extremely large, slow to train, and highly prone to overfitting.
- **Not Translation Invariant:** Because an MLP learns specific weights for specific input nodes, it is not inherently translation-invariant. If you train an MLP to recognize an object in the top-left corner of an image, it will struggle to recognize the same object if it appears in the bottom-right corner. It would have to be explicitly trained on images with the object in every possible position.
- **Local Feature Blindness:** MLPs look at all inputs globally and struggle to learn about local features efficiently. In contrast, CNNs are specifically designed to address these issues. They use convolutional layers with filters (kernels) that slide over the image to detect local features like edges, corners, and textures. This preserves spatial relationships, drastically reduces the number of parameters through weight sharing, and achieves translation invariance, making them far more effective for tasks like image recognition.

A typical CNN architecture



Introducing CNN

- **Specialized for Grid-Like Data:** CNNs are a class of neural networks specifically designed for processing data with a grid pattern, such as images. They excel where MLPs fall short by preserving the spatial relationships between pixels.
- **Feature Detection with Filters:** Instead of fully connected layers, CNNs use **convolutional layers** with filters (or kernels). These filters slide across the input image to detect specific local features, such as edges, corners, and textures, building a feature map.
- **Hierarchy and Invariance:** CNNs learn a hierarchy of features. Early layers detect simple patterns, which are combined by later layers to recognize more complex objects. This process makes them **translation invariant**, meaning they can recognize an object regardless of its position in the image.
- **Efficiency through Parameter Sharing:** Because the same filter is used across the entire image, CNNs share parameters. This drastically reduces the number of weights compared to an MLP, making them more computationally efficient and less susceptible to overfitting on image data.

How does CNN differ from MLP in design

Feature	Multi-Layer Perceptron (MLP)	Convolutional Neural Network (CNN)
Input Shape	1D vector (images must be flattened)	2D/3D grid/tensor (e.g., height x width x channels)
Connectivity	Fully connected layers	Local connectivity (neurons connect to a local receptive field)
Key Layers	Input, Hidden (Dense), Output	Convolutional, Pooling, Fully Connected (Dense)
Parameter Sharing	No parameter sharing (each connection has a unique weight)	Extensive parameter sharing (a single filter is used across the image)
Primary Strength	General-purpose function approximation	Learning spatial hierarchies and features in grid-like data

Convolution – the core in CNN

- **Sliding Filter (Kernel):** The operation involves a small matrix of weights called a **filter** or **kernel**. This filter slides or "convolves" across every possible location of the input image.
- **Element-wise Multiplication and Sum:** At each position, the filter is placed over a patch of the input. The network performs an element-wise multiplication between the values in the filter and the pixel values in the patch of the image it's currently covering. All the results of this multiplication are then summed up to produce a single pixel in the output, which is called a **feature map**.
- **Feature Detection:** The weights within the filter determine what kind of feature it is designed to detect. For example, one filter might be activated by horizontal edges, another by vertical edges, and another by a specific color or texture. The network learns the optimal values for these filters during the training process.

The convolution operation

For a 2D input like an image I and a 2D kernel K, the discrete convolution operation is defined as:

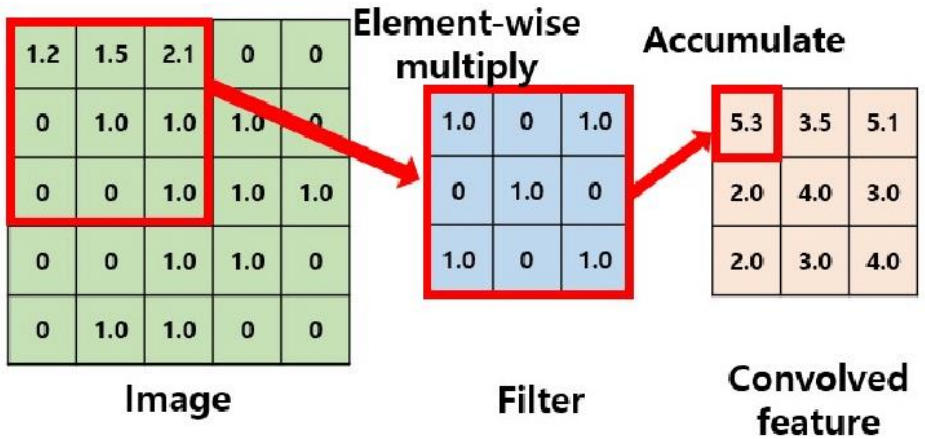
$$(I * K)(i,j) = \sum_m \sum_n I(m,n) \cdot K(i-m, j-n)$$

In practice, neural networks use a closely related operation called **cross-correlation**, which is computationally simpler and achieves the same goal since the kernel's weights are learned. The formula for cross-correlation is:

$$(I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

Here:

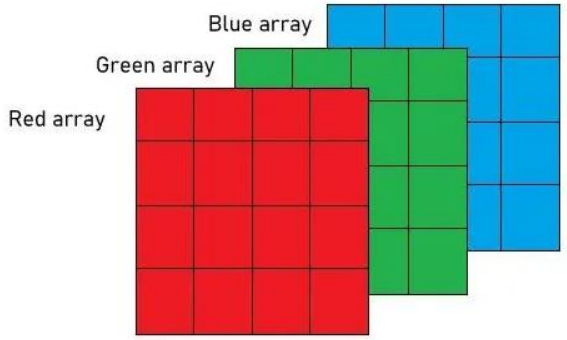
- (i,j) is the position of the output pixel in the feature map.
- (m,n) are the indices of the kernel.
- The sums are taken over all the elements of the kernel.

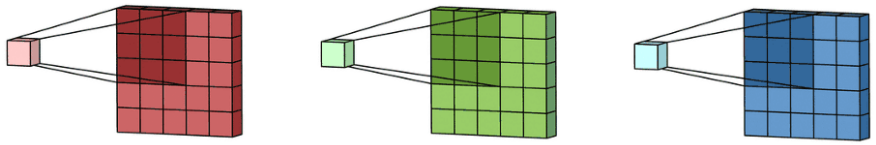


Remember! The kernels are learnable here!

- **Automated Feature Extraction:** The most powerful aspect of CNNs is that the kernels are **learnable**. Unlike traditional computer vision where filters were hand-engineered to detect specific features (e.g., a Sobel filter for edge detection), a CNN learns the optimal filter weights on its own during training.
- **Learning from Data:** The network starts with random values in its kernels. Through backpropagation, it adjusts these values to minimize the task's loss function. This means the network automatically discovers which features (edges, textures, shapes) are most important for making a correct prediction on the training data.
- **Hierarchical Learning:** Kernels in the early layers of a CNN learn to detect very basic features like simple edges and colors. Kernels in deeper layers learn to combine these simple features into more complex patterns, such as shapes, textures, and eventually entire object parts. This automated, hierarchical feature learning is what makes CNNs so effective.

Convolution on three channels

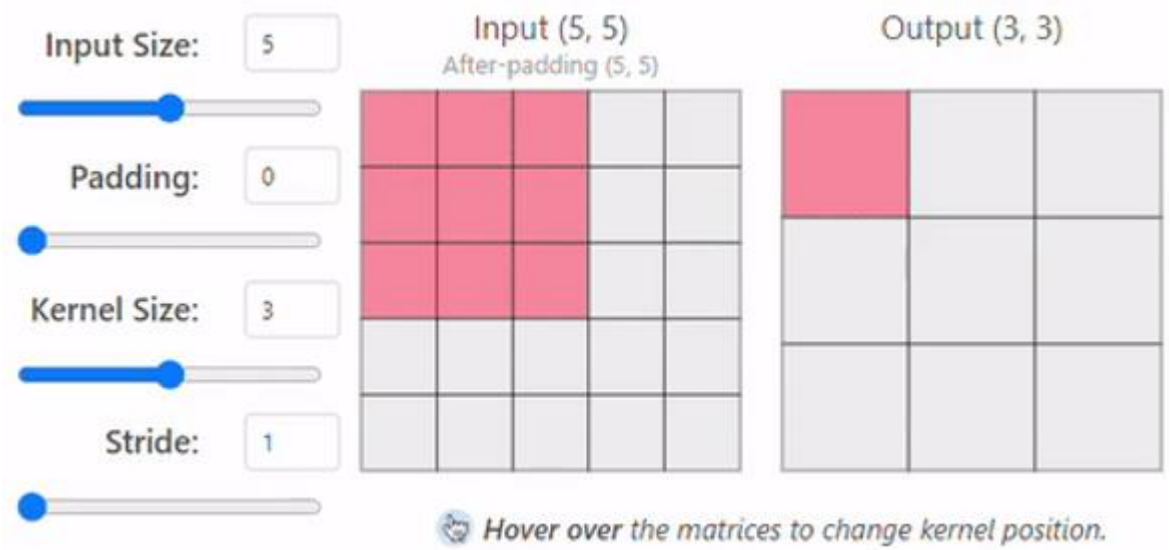


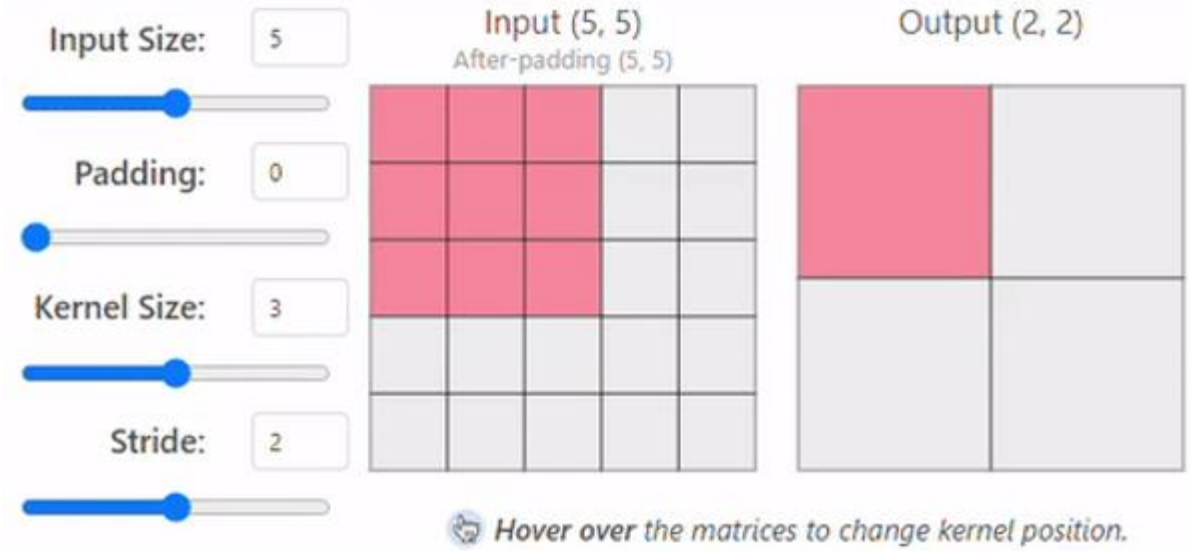


$$(I * K)(i,j) = b + \sum_{c=1}^{c_{in}} \sum_{m} \sum_{n} I(i+m, j+n, c) \cdot K(m,n,c)$$

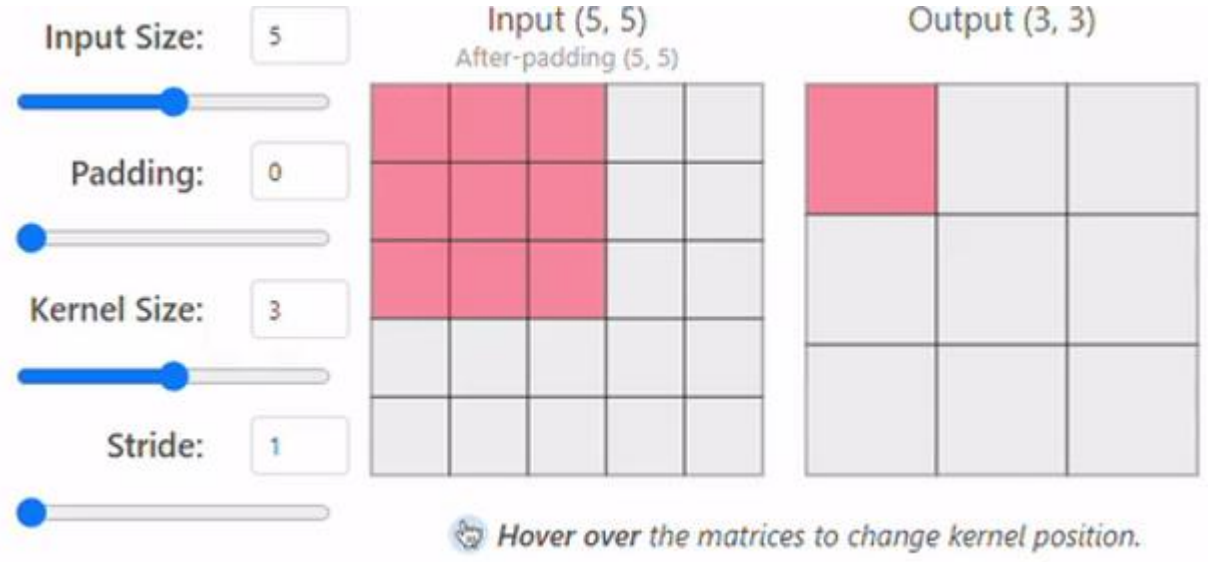
- **Channel Depth Matching:** The most important rule is that the filter (or kernel) must have the **same number of channels** as the input volume. So, for an RGB image (3 channels), the kernel will also have 3 channels (e.g., a 3x3x3 kernel). Each channel of the kernel is specialized to look for features in the corresponding channel of the input.
- **Channel-wise Operation:** The convolution is performed by sliding the multi-channel kernel over the input volume. At each position, each channel of the kernel is applied to its corresponding input channel (Red-kernel on Red-channel, etc.). This produces an intermediate result for each channel.
- **Summation to a Single Output Map:** After the element-wise multiplications are done for all channels, the results are **summed together** (along with a single bias term) to produce a single pixel value in the 2D output feature map. This means that even if your input has many channels, a single multi-channel filter will always produce a single-channel (2D) feature map. To get multiple feature maps, you apply multiple different filters.

Stride





Padding



How input and output sizes are related

$$\text{Output size} = \frac{\text{Input size} + 2 \times \text{Padding} - \text{Kernel size}}{\text{Stride}} + 1$$

Let's assume we have the following parameters:

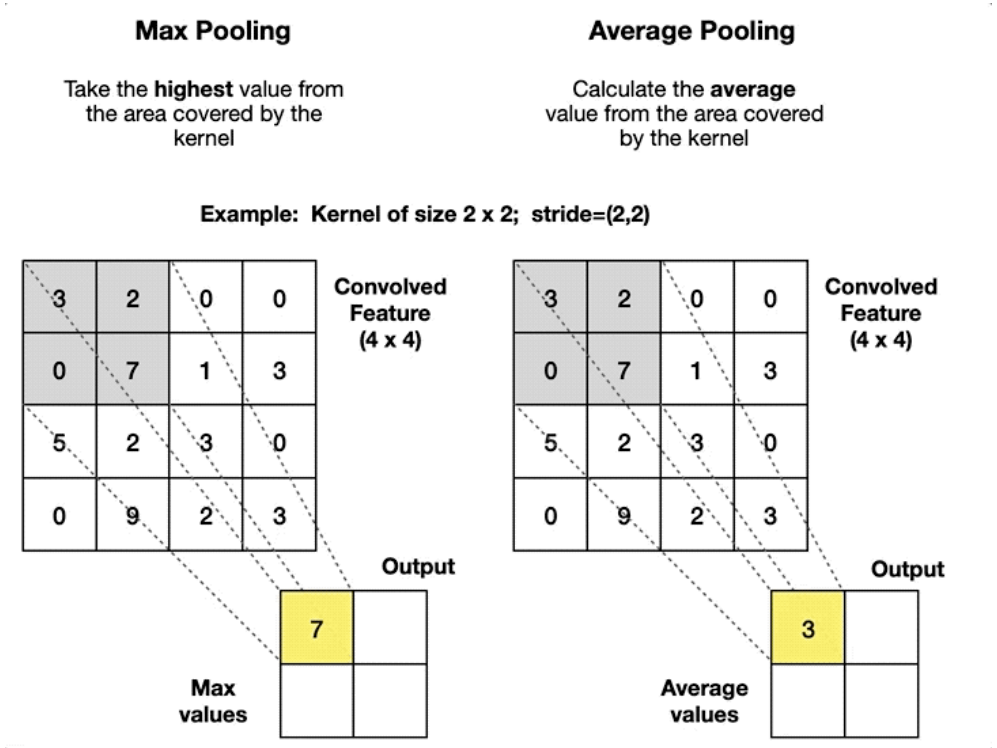
- **Input Volume Size ($W_{in} \times H_{in}$):** `227 x 227`
- **Kernel Size ($K_w \times K_h$):** `11 x 11`
- **Stride (S):** `4`
- **Padding (P):** `0` (no padding)

Calculation for Width (W_{out}):

1. $W_{out} = \lfloor \frac{227-11+2(0)}{4} \rfloor + 1$
2. $W_{out} = \lfloor \frac{216}{4} \rfloor + 1$
3. $W_{out} = \lfloor 54 \rfloor + 1$
4. $W_{out} = 54 + 1 = 55$

Since the parameters are symmetrical, the output height (H_{out}) is also **55.**

Pooling or downsampling layer



$$P_{out}(i,j) = \max_{m,n \in \mathcal{W}_{ij}} F_{in}(m,n)$$

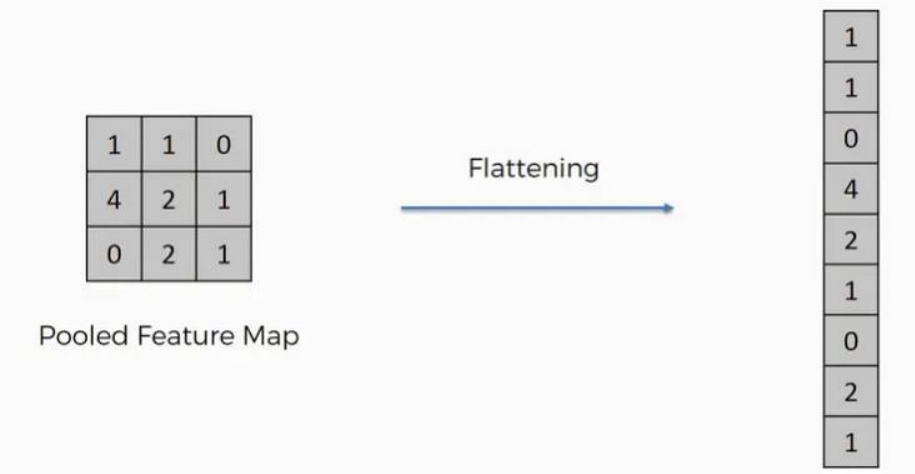
$$P_{out}(i,j) = \frac{1}{|\mathcal{W}_{ij}|} \sum_{m,n \in \mathcal{W}_{ij}} F_{in}(m,n)$$

- **Downsampling and Invariance:** The primary role of pooling is **downsampling**. It reduces the height and width of the feature maps, which leads to fewer parameters and computations in the network. This also helps in making the learned features more robust to small translations and distortions in the input image (translation invariance).
- **No Learnable Parameters:** Unlike convolutional layers, pooling layers have **no learnable parameters**. The operation is a fixed function (max or average). It only has hyperparameters like the window size and stride, which are defined when designing the network.

The effects of conv + pooling blocks

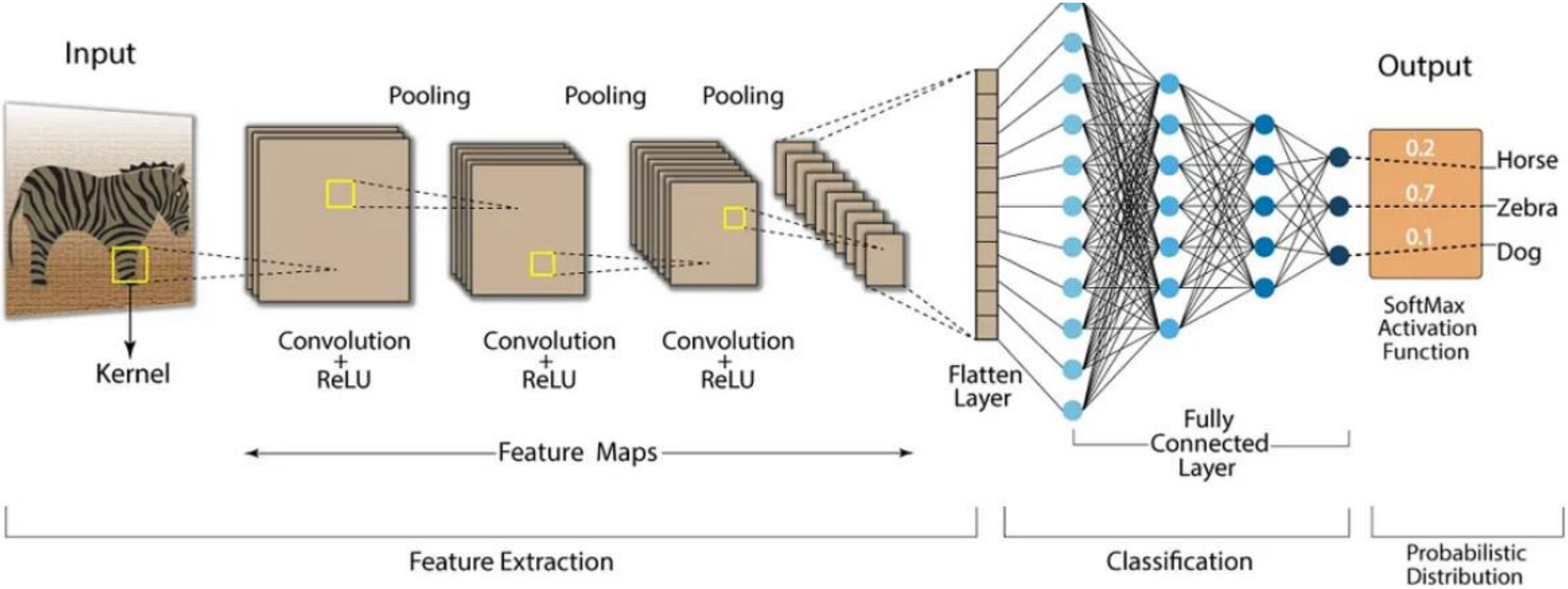
- **Hierarchical Feature Abstraction:** Convolutional layers operate in a hierarchy. The first layer learns to detect simple features like edges and corners. The next layer takes these feature maps as input and learns to combine the simple features into more complex ones, like shapes or textures. Stacking these layers allows the network to build an increasingly abstract and complex understanding of the image content.
- **Creating Local Invariance through Pooling:** After a convolutional layer detects a feature, the pooling layer downsamples the feature map. By taking the maximum (Max Pooling) or average value in a local patch, the pooling operation discards precise positional information. It essentially summarizes that a feature was present in a region, making the network's response less sensitive to the feature's exact location. This creates robustness to small shifts, rotations, and scaling.
- **Increasing the Receptive Field:** Every time you stack a convolutional or pooling layer, the neurons in the subsequent layer are connected to a larger effective region of the original input. This is called an expanding "receptive field." Deeper layers can therefore learn features based on a wider context, understanding how simpler parts (like an eye and a nose) are spatially related to form a more complex object (like a face).
- **Compositionality Leads to Global Invariance:** The combined effect of this stacking is that the network learns a representation that is both rich in features and invariant to their exact position. The convolution layers find the features, and the pooling layers make their representation more abstract and robust. By the final layers, the network is responding to the presence of complex object parts and their approximate arrangement, rather than their pixel-perfect locations, which is the essence of feature.

The MLP head for classification



1. **The Flattening Layer:**
   - **Bridge to the Classifier:** The Flatten layer acts as a bridge between the convolutional base and the MLP head.
   - **Reshaping Operation:** Its sole purpose is to take the final multi-dimensional feature map (e.g., 7x7x512) and unroll it into a single, long 1D vector. This is necessary because the fully connected layers of the MLP head expect a 1D vector as input.
   - **No Parameters:** This layer does not have any learnable parameters; it only reorders the data.
2. **The MLP Head (Fully Connected Classifier):**
   - **Final Classification:** This is a standard MLP, often consisting of one or more fully connected (Dense) layers, placed at the end of the CNN.
   - **Learning Feature Combinations:** It takes the flattened vector of high-level features and learns the non-linear combinations of these features that are predictive of the final classes.
   - **Output Layer:** The final layer of the MLP head has a neuron for each class. It typically uses a **Softmax activation function** to output a probability distribution over the classes, indicating the network's confidence for each one.

The full picture



What are the normalization layers?

- **Accelerating & Stabilizing Training:** Normalization helps combat a problem called "internal covariate shift," where the distribution of inputs to layers changes during training. By standardizing the activations (e.g., to have zero mean and unit variance), it ensures a much more stable learning process. This allows for the use of higher learning rates, which significantly speeds up training.
- **Regularization Effect:** The statistics (mean/variance) used for normalization are calculated on mini-batches of data. This introduces a slight noise into the network, which acts as a form of regularization. This can reduce the model's reliance on other techniques like Dropout and help prevent overfitting.
- **Reducing Sensitivity to Initialization:** By re-centering and re-scaling the activations at each layer, the network becomes far less sensitive to the choice of initial weights. This makes the training process more robust and the results easier to reproduce.

Batch-normalization

- **Normalization Per Mini-Batch:** As its name suggests, Batch Normalization calculates the mean and variance for the current mini-batch of data. It then uses these statistics to normalize the activations for that batch. This process standardizes the inputs to the next layer, ensuring they have a stable distribution (typically zero mean and unit variance).
- **Combating Internal Covariate Shift:** This is the core problem Batch Norm solves. As the weights in earlier layers change during training, the distribution of data flowing to later layers also shifts. By constantly re-normalizing the data per batch, it provides a more consistent signal to subsequent layers, which stabilizes and accelerates the learning process.
- **Learnable Scaling and Shifting:** Batch Norm doesn't just blindly standardize the data. It introduces two learnable parameters per feature: a scaling factor (**gamma**, γ) and a shifting factor (**beta**, β). This allows the network to learn the optimal scale and mean for the activations, giving it the flexibility to undo the normalization if that's what's best for the learning task.
- **Inference Time Behavior:** During testing or inference, there are no mini-batches. Instead, Batch Norm uses a running average of the mean and variance it calculated across all the mini-batches during training. This provides a stable, estimated distribution to normalize the activations for a single input.

Setting: CNN feature maps of shape (N, C, H, W)

- N = batch size
- C = channels
- H, W = spatial dimensions

Stages:

1. **Compute Mean (per channel, across batch and spatial dimensions):**

   For each channel c:

$$\mu_c = \frac{1}{N \cdot H \cdot W} \sum_{n=1}^{N} \sum_{h=1}^{H} \sum_{w=1}^{W} x_{n,c,h,w}$$

2. **Compute Variance (per channel, across batch and spatial dimensions):**

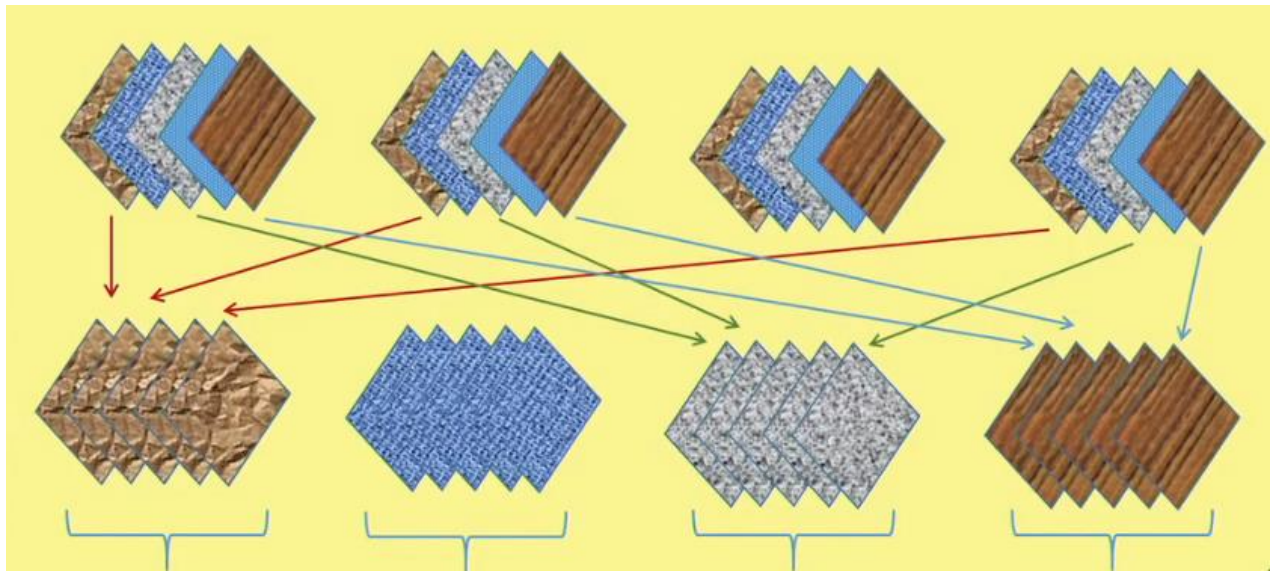
$$\sigma_c^2 = \frac{1}{N \cdot H \cdot W} \sum_{n,h,w} (x_{n,c,h,w} - \mu_c)^2$$

3. **Normalize:**

$$\hat{x}_{n,c,h,w} = \frac{x_{n,c,h,w} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}$$

4. **Scale and Shift (learnable parameters):**

$$y_{n,c,h,w} = \gamma_c \cdot \hat{x}_{n,c,h,w} + \beta_c$$



Advantages of BN

- **Faster Convergence:** By normalizing the activations, Batch Norm ensures that the distribution of inputs to each layer remains stable during training. This prevents the learning process from getting stuck or slowing down, which in turn allows you to use a higher learning rate, leading to significantly faster training times.
- **Improved Gradient Flow:** The normalization helps prevent issues like vanishing or exploding gradients. By keeping the activations in a reasonable range, it ensures that the gradients flowing backward through the network are more stable, making the training process more reliable, especially in deep networks.
- **Regularization to Prevent Overfitting:** Because the normalization statistics (mean and variance) are calculated on small, random mini-batches of data, it introduces a slight amount of noise into the training process. This noise acts as a form of regularization, making the model more robust and reducing its tendency to overfit the training data. This can sometimes reduce or eliminate the need for other regularization techniques like Dropout.

Replacing BN by Layer normalization

- **High Dependency on Batch Size:** Batch Norm's effectiveness is strongly tied to the size of the mini-batch. If the batch size is too small (e.g., less than 8-16), the calculated mean and variance become noisy and are not a good estimate of the overall data distribution. This can hurt the model's performance and make training unstable.
- **Problems with Sequence Data (RNNs/Transformers):** BN is difficult to apply effectively to recurrent neural networks (RNNs) and other sequence-based models. Input sequences can have variable lengths, which makes it challenging to compute consistent batch statistics across the time-step dimension.
- **Discrepancy Between Training and Inference:** BN behaves differently during training (where it calculates statistics per batch) and inference (where it uses fixed running averages). This can sometimes lead to a slight performance drop when the model is deployed, as the running averages may not perfectly represent the true data distribution.

- Layer Normalization was created to address these specific issues.
- **How it Works:** Instead of normalizing across the batch dimension, Layer Norm normalizes across the **feature dimension** for a single training example. It calculates the mean and variance from all the activations within a single layer for one data point.
- **Solving the Problems:**
  - **It is independent of the batch size**, making it suitable for any batch size, including 1.
  - It works very well for **RNNs and Transformers** because it normalizes each sequence in the batch independently.
  - It behaves **identically during training and inference**, removing the discrepancy that exists in Batch Norm.

Suppose we have **1 image** after flattening (or one hidden representation vector in a Transformer).

Let's say the feature vector is:

$$x = [2,\ 4,\ 6,\ 8]$$

Here, feature dimension $D = 4$.
(LayerNorm operates **across features of a single sample**).

🔷 Stagewise Steps

1. Compute Mean across Features

$$\mu = \frac{1}{D}\sum_{d=1}^{D} x_d = \frac{2+4+6+8}{4} = \frac{20}{4} = 5$$

---

2. Compute Variance across Features

$$\sigma^2 = \frac{1}{D}\sum_{d=1}^{D}(x_d - \mu)^2$$

$$= \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4}$$

$$= \frac{9+1+1+9}{4} = \frac{20}{4} = 5$$

3. Normalize Each Feature

$$\hat{x}_d = \frac{x_d - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Let's ignore ϵ for simplicity.

- For $x_1 = 2$: $\hat{x}_1 = \frac{2-5}{\sqrt{5}} = \frac{-3}{2.236} \approx -1.34$
- For $x_2 = 4$: $\hat{x}_2 = \frac{4-5}{\sqrt{5}} = -0.45$
- For $x_3 = 6$: $\hat{x}_3 = \frac{6-5}{\sqrt{5}} = +0.45$
- For $x_4 = 8$: $\hat{x}_4 = \frac{8-5}{\sqrt{5}} = +1.34$

So the normalized vector:

$$\hat{x} \approx [-1.34,\ -0.45,\ +0.45,\ +1.34]$$

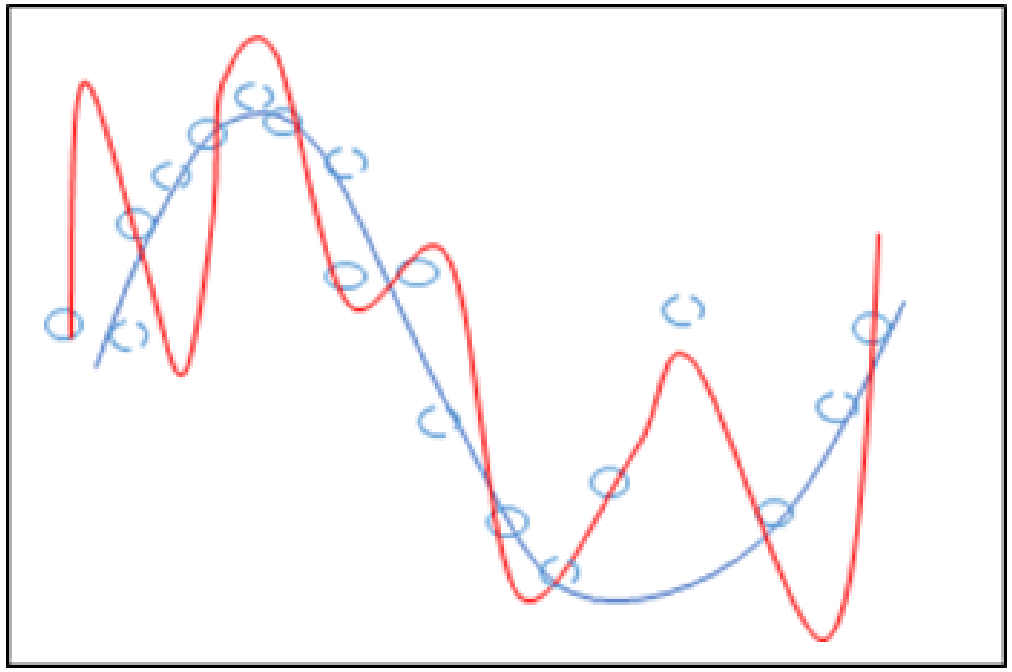
---

4. Apply Scale and Shift (γ, β)

LayerNorm also has learnable parameters:

$$y_d = \gamma_d \cdot \hat{x}_d + \beta_d$$

Model regularization

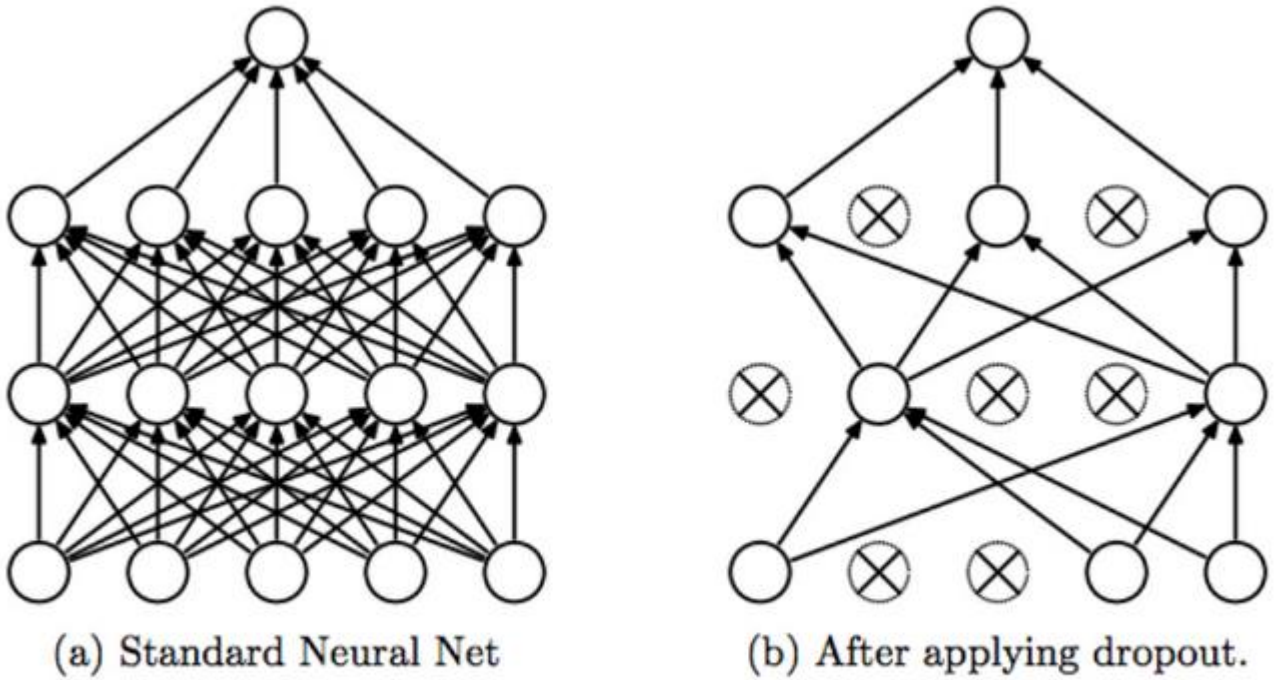


1. Less training data
2. Too many model parameters
3. Model overfitting
4. Regularization – ways to avoid overfitting

Regularization ways

- We have to ensure that the model does not overfit the training data – does not memorize the training samples
- Restrict the values the model parameters can take (lasso, ridge regressions)
- Ensemble model – mixture of experts

Dropout regularization



(a) Standard Neural Net (b) After applying dropout.

Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting", JMLR 2014

Given

- $h(x) = xW + b$ a linear projection of a d_i-dimensional input x in a d_h-dimensional output space.
- $a(h)$ an activation function

it's possible to model the application of Dropout, in the training phase only, to the given projection as a modified activation function:

$$f(h) = D \odot a(h)$$

Where $D = (X_1, \cdots, X_{d_h})$ is a d_h-dimensional vector of Bernoulli variables X_i.

A Bernoulli random variable has the following probability mass distribution:

$$f(k; p) = \begin{cases} p & \text{if} \quad k = 1 \\ 1 - p & \text{if} \quad k = 0 \end{cases}$$

Where k are the possible outcomes.

$$O_i = X_i a(\sum_{k=1}^{d_i} w_k x_k + b) = \begin{cases} a(\sum_{k=1}^{d_i} w_k x_k + b) & \text{if} \quad X_i = 1 \\ 0 & \text{if} \quad X_i = 0 \end{cases}$$

where $P(X_i = 0) = p$.

Suppose we have a small hidden layer with activations (before dropout):

$$h = [0.2,\ 0.8,\ -0.5,\ 1.0]$$

We apply **Dropout with probability** $p = 0.5$ (i.e., each neuron is kept with probability 0.5).

🔷 Stagewise Example

1. Generate Dropout Mask

Dropout samples a random **binary mask** for each forward pass.

Each neuron has a 50% chance of being "kept" (1) or "dropped" (0).

Example mask:

$$m = [1,\ 0,\ 1,\ 0]$$

---

2. Apply Mask

Element-wise multiply:

$$h' = h \odot m = [0.2,\ 0,\ -0.5,\ 0]$$

```
class SmallCNN(nn.Module):
    def __init__(self):
        super(SmallCNN, self).__init__()
        # Conv layer + BatchNorm
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.bn1   = nn.BatchNorm2d(16)   # batch norm after conv

        # Another Conv + BatchNorm
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.bn2   = nn.BatchNorm2d(32)

        # Fully connected + Dropout
        self.fc1   = nn.Linear(32*8*8, 128)
        self.drop  = nn.Dropout(p=0.5)    # dropout before final layer
        self.fc2   = nn.Linear(128, 10)   # 10 classes
```

	$M=0$	$M=1$	$M=3$	$M=9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$w_5^\star$				640042.26
$w_6^\star$				-1061800.52
$w_7^\star$				1042400.18
$w_8^\star$				-557682.99
$w_9^\star$				125201.43

Dropout + L2 normalization

$$\mathcal{L}(y, \hat{y}) = \mathcal{E}(y, F(W; x)) + \frac{\lambda}{2} W^2$$

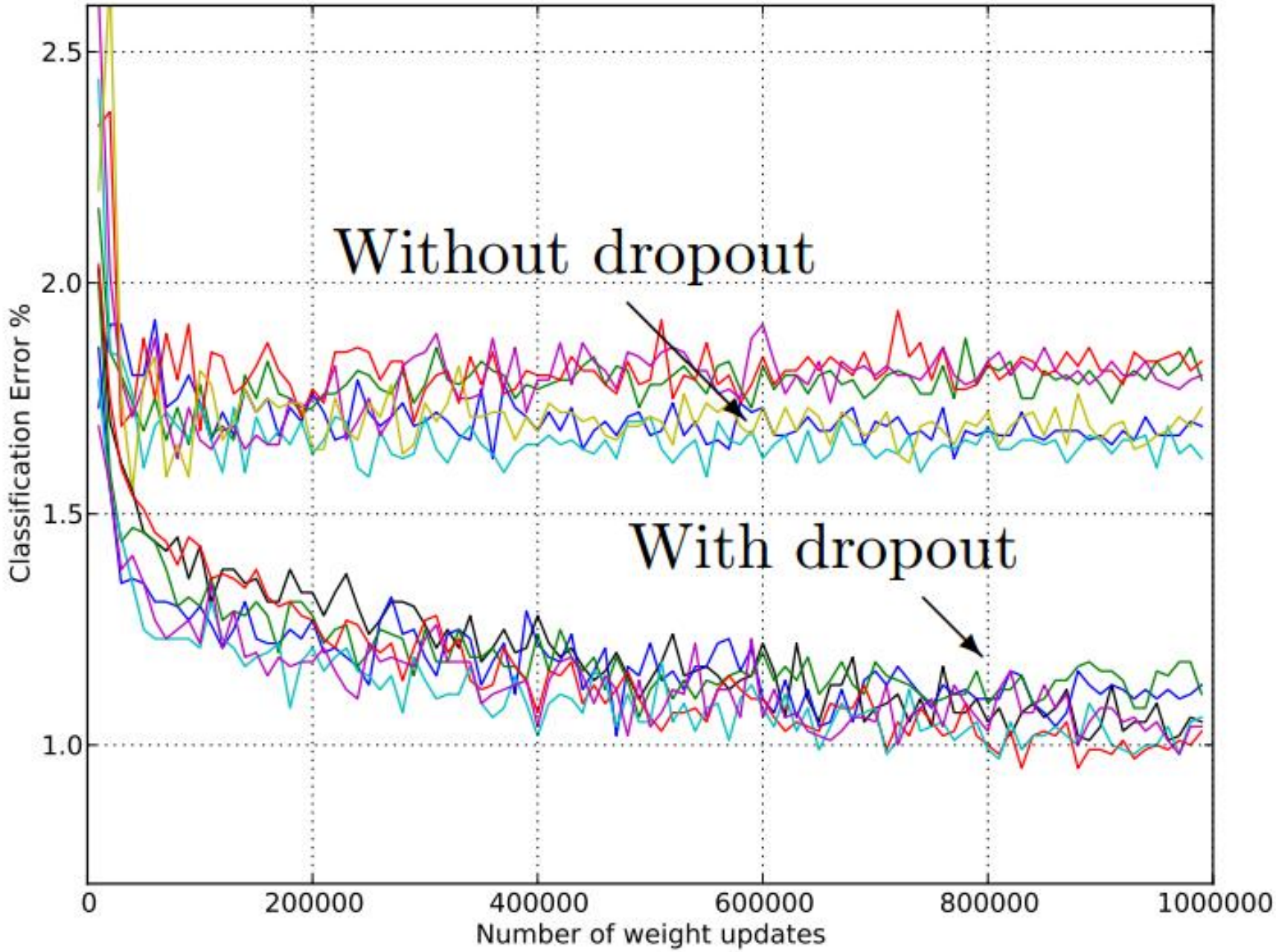
Dropout can’t control the range of values the weight variables can take!

Dropout

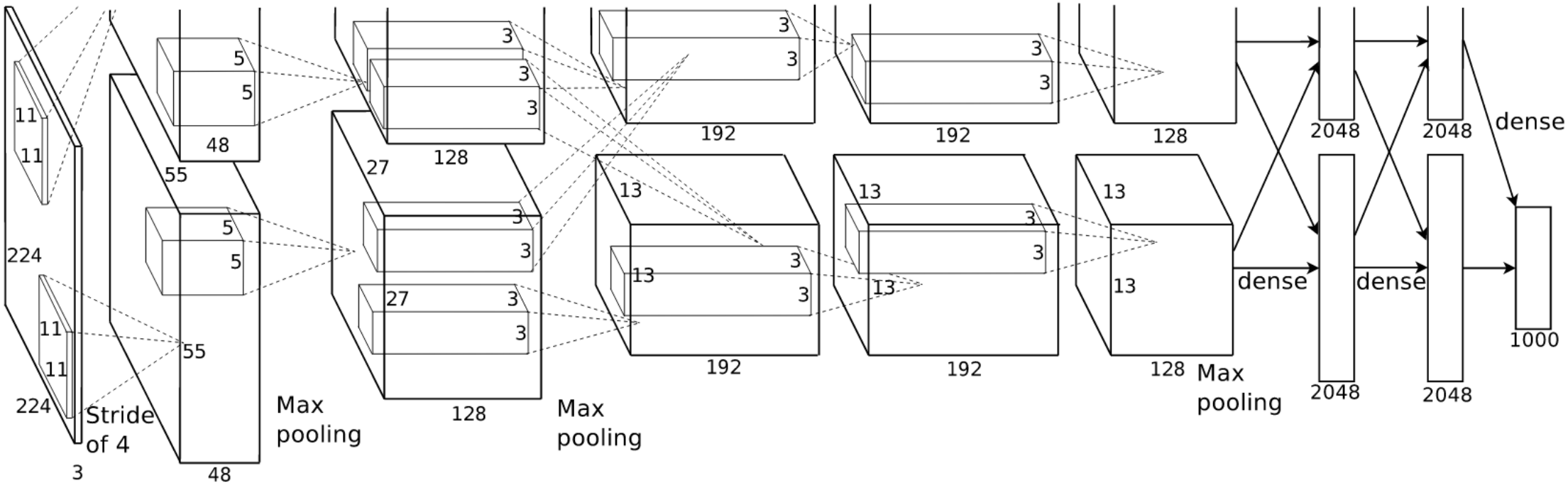
$$w \leftarrow w - \eta \left(\frac{\partial F(W; x)}{\partial w} + \lambda w \right)$$

Inverted Dropout

$$w \leftarrow w - \eta \left(\frac{1}{q} \frac{\partial F(W; x)}{\partial w} + \lambda w \right)$$



AlexNet



Salient features

- **Architecture Overview**
  - **Depth:** 8 layers (5 convolutional + 3 fully connected).
  - **Convolutions:** Extract hierarchical features (edges → textures → objects).
  - **Fully Connected Layers:** Perform classification on extracted features.
  - **Output:** Softmax over 1000 ImageNet classes.

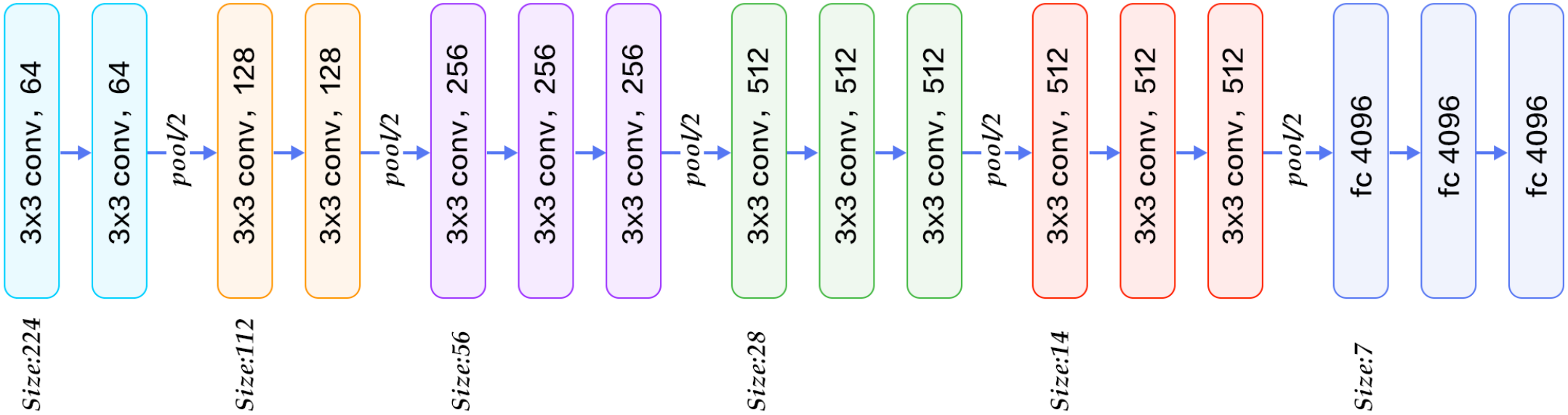
- **Key Innovations**
  - **ReLU Activation:** Faster training and alleviation of vanishing gradient problem.
  - **GPU Utilization:** First large-scale CNN trained efficiently using GPUs.
  - **Dropout Regularization:** Prevented overfitting in fully connected layers.
  - **Data Augmentation:** Used random cropping, flipping, and color jittering to increase dataset variability.
  - **Local Response Normalization (LRN):** Encouraged competition among neurons (though less used today).

- **Technical Details**
  - **Input size:** 224×224 RGB image.
  - **Filters:** First layer used large 11×11 kernels with stride 4 (later reduced in newer architectures).
  - **Pooling:** Overlapping max pooling (stride 2, kernel 3×3).
  - **Parameters:** ~60 million parameters, ~650,000 neurons.
  - **Training:** Stochastic Gradient Descent (SGD) with momentum and weight decay.

```python
class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        # Feature extractor (conv → ReLU → LRN → pool, etc.)
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),   # 224→55
            nn.ReLU(inplace=True),
            nn.LocalResponseNorm(size=5, alpha=1e-4, beta=0.75, k=2.0),
            nn.MaxPool2d(kernel_size=3, stride=2),                   # 55→27

            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.LocalResponseNorm(size=5, alpha=1e-4, beta=0.75, k=2.0),
            nn.MaxPool2d(kernel_size=3, stride=2),                   # 27→13

            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),                   # 13→6
        )
```

```python
self.classifier = nn.Sequential(
    nn.Dropout(p=0.5),
    nn.Linear(256 * 6 * 6, 4096),
    nn.ReLU(inplace=True),

    nn.Dropout(p=0.5),
    nn.Linear(4096, 4096),
    nn.ReLU(inplace=True),

    nn.Linear(4096, num_classes),
)
```

VGG-Net

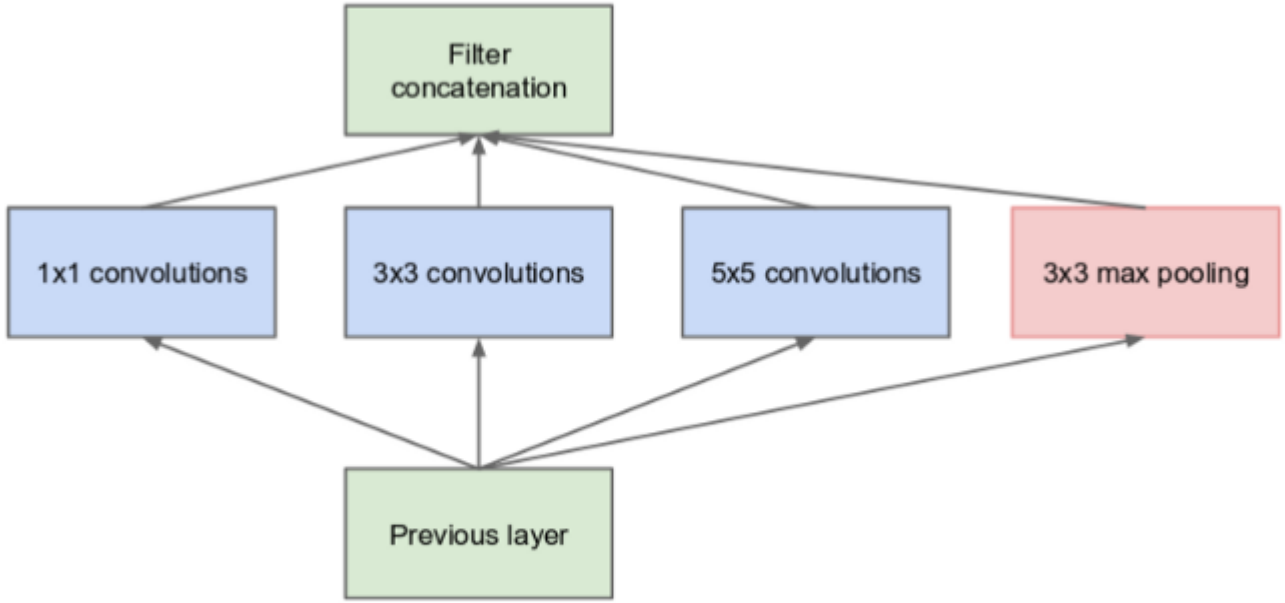


Salient features

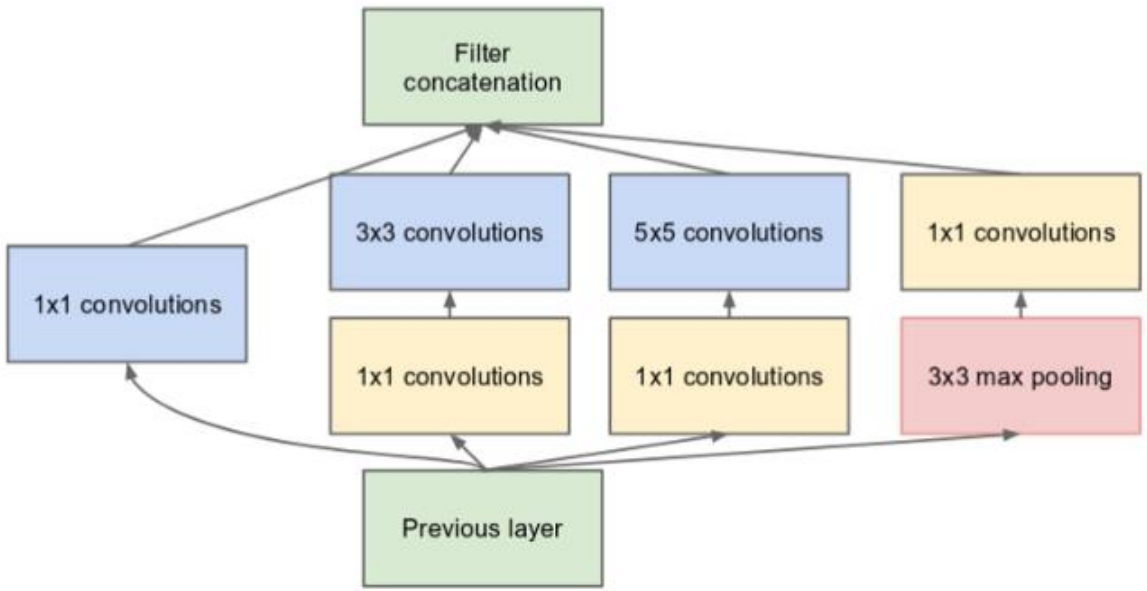
- **Architecture Highlights**
  - **Depth:** Evaluated variants from 11 to 19 layers (VGG16 and VGG19 are most popular).
  - **Convolutions:** All convolution layers use **3×3 kernels** (stride 1, padding 1).
    - 3×3 chosen as the smallest receptive field that captures left/right, up/down relationships.
    - Multiple stacked 3×3 convs emulate larger receptive fields (e.g., two 3×3 ≈ one 5×5).
  - **Pooling:** 2×2 max-pooling with stride 2.
  - **Fully Connected Layers:** Three dense layers at the end (two 4096-dim + 1000-way softmax).
- **Key Innovations**
  - Showed that **network depth is critical** for better performance.
  - **Uniform architecture:** Very consistent design (only 3×3 convs + 2×2 pooling), unlike AlexNet's mixed kernel sizes.
  - **Increased non-linearity:** Multiple small convs provide more ReLU activations compared to larger filters.
  - **Parameter efficiency:** Stacking small filters reduces parameters while keeping receptive field size (e.g., 3×3 + 3×3 = 5×5 receptive field but fewer parameters than a single 5×5 conv).
- **Technical Details**
  - **Input:** 224×224 RGB images.
  - **Feature maps:** Start with 64 channels, doubling after each pooling (64 → 128 → 256 → 512).
  - **Parameters:** ~138 million (VGG16), making it heavy in memory and computation.
  - **Training:** SGD with momentum, weight decay, and data augmentation.

Inception architecture

What is a good kernel size for DOG feature extraction?

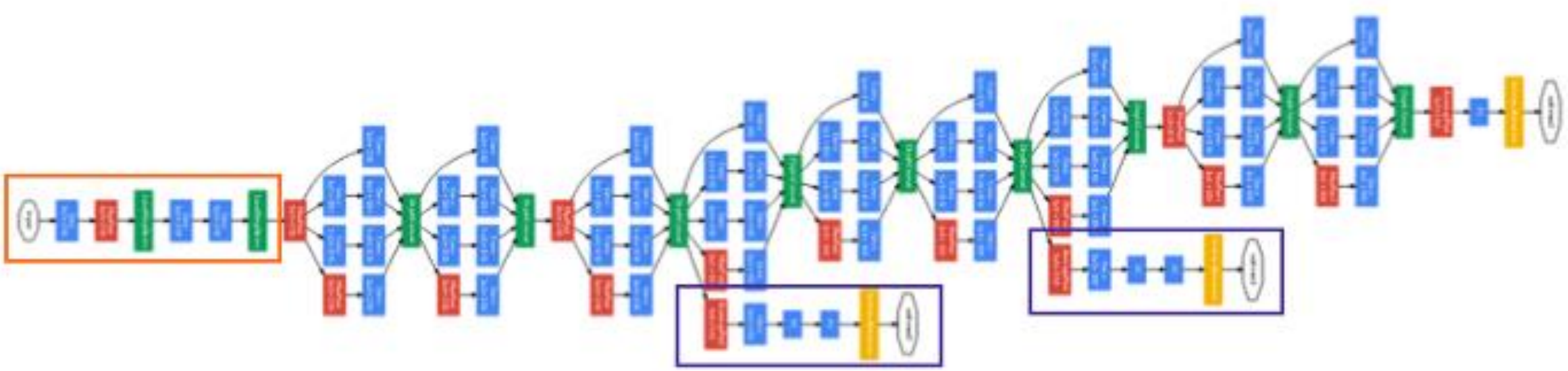


(a) Inception module, naïve version



(b) Inception module with dimension reductions

Inception v1 (Googlenet)



```
# The total loss used by the inception net during training.
total_loss = real_loss + 0.3 * aux_loss_1 + 0.3 * aux_loss_2
```

9 inception module, 22 layers, global average pooling at the end, 2 intermediate classifier to take care
Of the ‘dying out’ effect

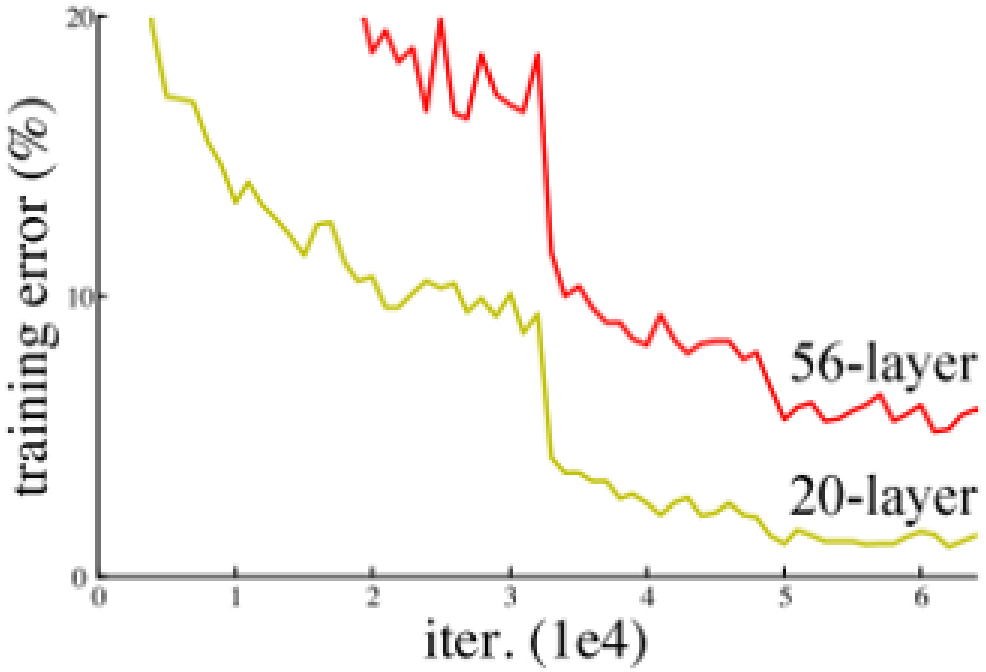
Core ideas

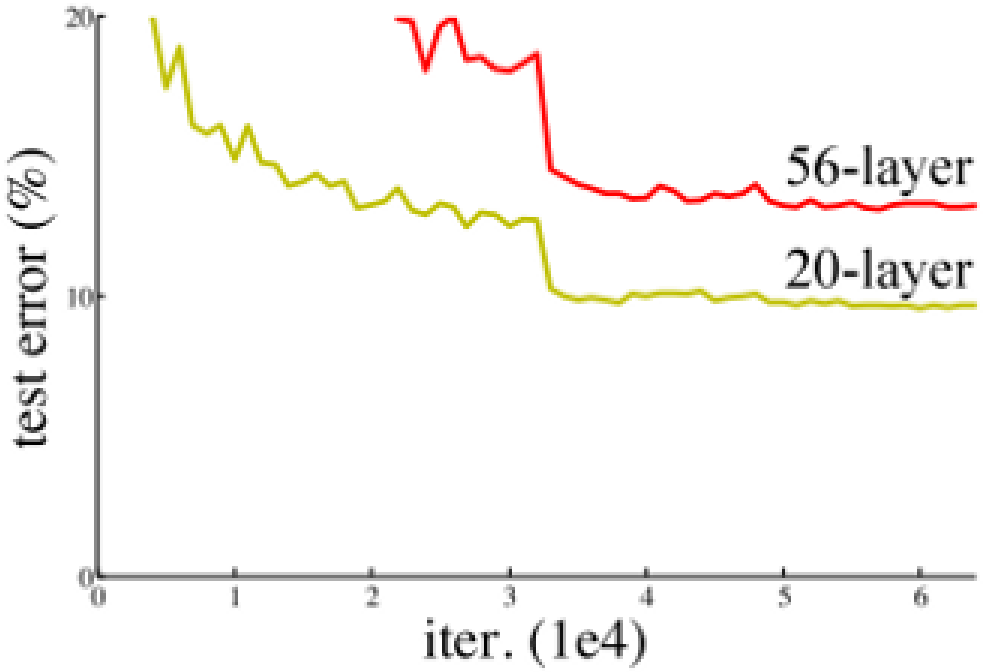
- **Inception module (multi-branch):** Parallel paths with 1×1, 3×3, 5×5 (or factorized) convolutions and pooling; outputs concatenated → rich multi-scale features at the same spatial resolution.
- **1×1 bottlenecks:** Use 1×1 convs before costlier 3×3/5×5 to reduce channel depth (parameters/FLOPs) while adding nonlinearity.
- **Parameter efficiency:** Much fewer parameters than contemporaries (e.g., GoogLeNet ~5–7M vs VGG16 ~138M) with higher accuracy.
- **Global average pooling:** Replaces large fully connected heads → fewer params, less overfitting.
- **Auxiliary classifiers (training only):** Intermediate heads provide extra gradient signal and regularization; dropped at inference.

Architectural refinements across versions

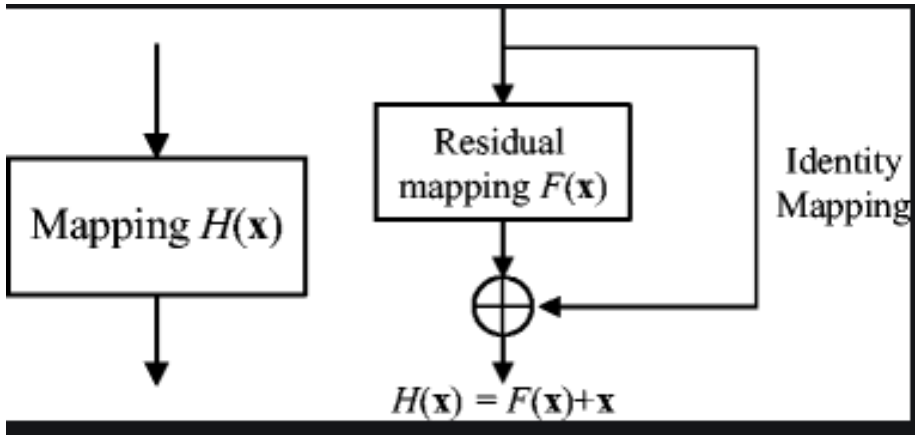
- **Inception-v1 (GoogLeNet, 2014):**
  - First multi-branch Inception blocks.
  - Overlapping pooling and 1×1 reductions; auxiliary losses.
- **Inception-v2 / v3 (2015–16):**
  - **Batch Normalization** throughout for faster, more stable training.
  - **Factorized convolutions:**
    - 5×5 → two 3×3;
    - large k×k → **1×k + k×1** (e.g., 7×7 → 1×7 then 7×1) to cut FLOPs and parameters.
  - **Efficient grid-size reduction:** Carefully designed stride/pooling combos to downsample without bottlenecking channels.
  - Training tricks: label smoothing, RMSProp (commonly used), tuned dropout.
- **Inception-v4 / Inception-ResNet (2016):**
  - Deeper, cleaner **stem**; standardized **Inception-A/B/C** and **Reduction-A/B** blocks.
  - **Residual connections** (Inception-ResNet) → easier optimization at even greater depth.

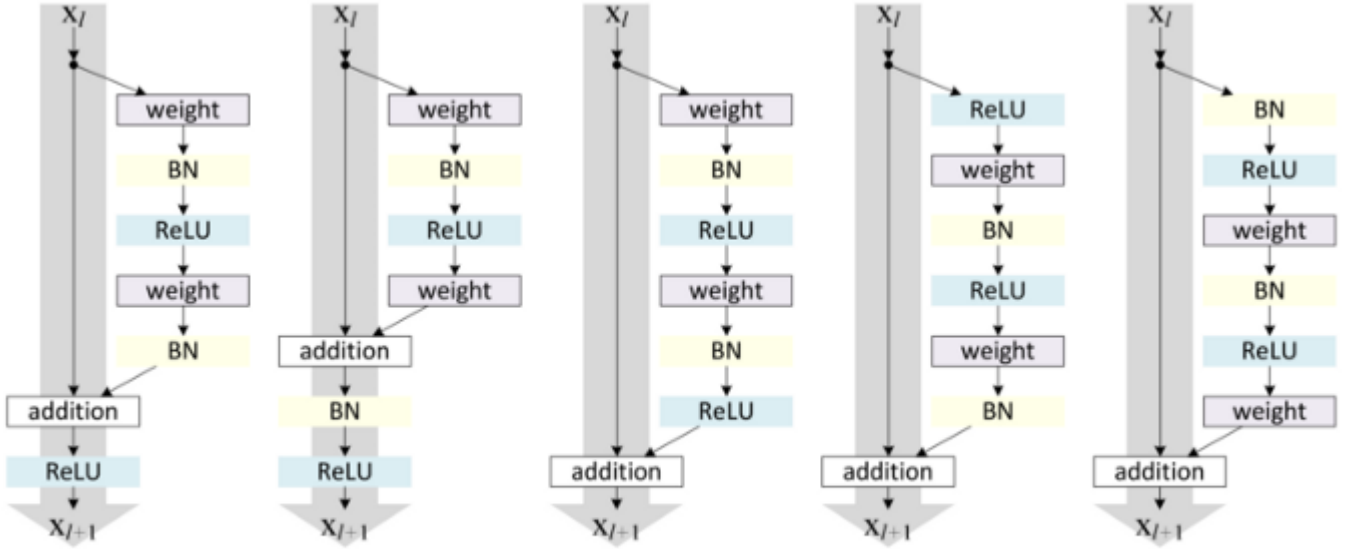
Going deeper in CNN





Residual networks





```python
class ResidualBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding=1, bias=False)
        self.bn1   = nn.BatchNorm2d(out_ch)
        self.relu  = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(out_ch)

        # projection (1x1) only if shape changes; else identity
        self.proj = None
        if stride != 1 or in_ch != out_ch:
            self.proj = nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False)

    def forward(self, x):
        identity = x if self.proj is None else self.proj(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = self.relu(out + identity)
        return out
```

Some salient features

- **Core idea**
  - **Residual learning:** Layers learn a *residual* F(x)F(x)F(x) w.r.t. the identity, outputting y=F(x)+x. This preserves signal/gradients and cures the **degradation problem** (deeper nets performing worse).
- **Skip connections**
  - **Identity shortcuts** by default (no extra params/FLOPs).
  - **Projection shortcuts** (1×1 conv, often stride 2) used when dimensions change.
  - Enable **unimpeded gradient flow**, better optimization, and robustness to vanishing gradients.
- **Building blocks**
  - **Basic block (ResNet-18/34):** 3×3 → 3×3 with residual add.
  - **Bottleneck block (ResNet-50/101/152):** 1×1 (reduce) → 3×3 (process) → 1×1 (expand); parameter/FLOP efficient at depth.
  - **Downsampling:** via stride-2 in the first conv of a block (and matching shortcut).
- **Normalization & activation**
  - **BatchNorm + ReLU** throughout.
  - **Pre-activation (ResNet v2):** BN → ReLU → Conv ordering improves optimization, enables very deep nets.
- **Depth & scalability**
  - Scales to **50–152+ layers** with strong accuracy and manageable compute.
  - Modular design makes it easy to widen, deepen, or add attention/se blocks.

Densely connected CNN